```
using DalApi;
using DO;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Xml.Linq;

namespace Dal
{
    internal class CustomerImplementation : ICustomer
    {
        // הם בסיומת .xml שימי לב לנתיב - ודאי שהקבצים בתיקיית xml .xml ולא .xml.txt
        private static string path = @"..\xml\customer.xml";

        const string CUSTOMER = "Customer";
        const string CUSTOMER_ID = "CustomerId";
        const string CUSTOMER_NAME = "CustomerName";
        const string ADDRESS = "Address";
        const string PHONE = "Phone";

        public int Create(Customer item)
        {
            XElement file = XElement.Load(path);

            XElement? customer = file.Descendants(CUSTOMER)
                .FirstOrDefault(c => int.Parse(c.Element(CUSTOMER_ID).Value) == item.id);

            if (customer != null)
                throw new Exception($"Customer with ID {item.id} already exists");

            file.Add(new XElement(CUSTOMER,
                new XElement(CUSTOMER_ID, item.id),
                new XElement(CUSTOMER_NAME, item.name),
                new XElement(ADDRESS, item.adress),
                new XElement(PHONE, item.phone)));

            file.Save(path);
            return item.id;
        }

        public void Delete(int id)
        {
            XElement file = XElement.Load(path);

            XElement? customer = file.Descendants(CUSTOMER)
                .FirstOrDefault(c => int.Parse(c.Element(CUSTOMER_ID).Value) == id);

            if (customer == null)
                throw new Exception("Customer not found to delete");
```

```
            customer.Remove();
            file.Save(path);
        }

        public Customer? Read(int id)
        {
            XElement file = XElement.Load(path);

            XElement? c = file.Descendants(CUSTOMER)
                .FirstOrDefault(c => int.Parse(c.Element(CUSTOMER_ID).Value) == id);

            if (c == null) return null;

            return new Customer(
                int.Parse(c.Element(CUSTOMER_ID).Value),
                c.Element(CUSTOMER_NAME).Value,
                c.Element(ADDRESS).Value,
                int.Parse(c.Element(PHONE).Value)
            );
        }

        public Customer? Read(Func<Customer, bool> filter)
        {
            return ReadAll().FirstOrDefault(filter!);
        }

        public IEnumerable<Customer?> ReadAll(Func<Customer, bool>? filter = null)
        {
            XElement file = XElement.Load(path);

            var customers = file.Descendants(CUSTOMER).Select(c => (Customer?)new Customer(
                int.Parse(c.Element(CUSTOMER_ID).Value),
                c.Element(CUSTOMER_NAME).Value,
                c.Element(ADDRESS).Value,
                int.Parse(c.Element(PHONE).Value)
            ));

            if (filter == null)
                return customers;

            return customers.Where(c => c != null && filter(c));
        }

        public void Update(Customer item)
        {
            XElement file = XElement.Load(path);

            XElement? customer = file.Descendants(CUSTOMER)
                .FirstOrDefault(c => int.Parse(c.Element
```

```
                    (CUSTOMER_ID).Value) == item.id);

            if (customer == null)
                throw new Exception("Customer not found to update");

            customer.Element(CUSTOMER_NAME).SetValue(item.name);
            customer.Element(ADDRESS).SetValue(item.adress ?? "");
            customer.Element(PHONE).SetValue(item.phone);

            file.Save(path);
        }
    }
}

```